Technical documentation: Pre-lab CKD Early Detection using Random Forest

Author: Antonio Garcia

1. Purpose

This project is a screening tool for chronic kidney disease (CKD). It uses only information a health worker can collect without a laboratory (age, sex, weight, height, waist, blood pressure, and a few yes/no health questions) and recommends whether a person should be referred for kidney testing (serum creatinine and urine albumin-to-creatinine ratio). It is meant for adults aged 20 and over who have never been diagnosed with kidney disease. It does not diagnose CKD.

The model is trained on NHANES 2017-March 2020 (US CDC). A second dataset, UCI Chronic Kidney Disease, is used to show how much of the near-perfect accuracy reported in the literature comes from features that are part of the diagnosis itself.

2. Requirements and installation

- Python 3.12
- The packages in `requirements.txt` (pinned versions). `requirements-dev.txt` adds the tools for running the tests and rebuilding the notebooks and documents.

Windows (PowerShell):

```
git clone https://github.com/2nieGarcia/intro-ml-final.git
cd intro-ml-final
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
```

macOS / Linux:

```
git clone https://github.com/2nieGarcia/intro-ml-final.git
cd intro-ml-final
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
```

Use `requirements.txt` instead of `requirements-dev.txt` if you only want to run the app.

3. Folder structure

```
app/streamlit_app.py          Streamlit app
.streamlit/config.toml        App theme (colors and font)
data/original/nhanes/         10 original NHANES files (.xpt)
data/original/uci/            Original UCI CKD file (.csv)
data/original/SOURCE.md       Source, license and download notes
data/processed/               nhanes_cohort.csv (cohort, label, features, train/test split)
```

```
notebooks/01_nhanes_eda.ipynb  NHANES data quality checks and EDA
notebooks/02_uci_ablation.ipynb UCI tier ablation
notebooks/03_nhanes_modeling.ipynb  Model comparison, selection, final test, saving the pipeline
src/download_data.py           Downloads the original data
src/nhanes.py                  NHANES cohort rules, eGFR equation, label, features
src/uci.py                     UCI cleaning and feature tiers
src/modeling.py                Shared preprocessing, tuning, cross-validation and metrics
src/screening.py               Input validation and prediction used by the app
src/plot_style.py              Chart colors and fonts
models/                        Saved pipeline, model card and result tables
tests/                         Automated tests (pytest)
documentation/                 Pre-registration, data dictionary, this document, figures,
screenshots
paper/                         IMRaD paper (DOCX and PDF)
```

4. How to run

All commands are run from the project root with the virtual environment active.

Task	Command
Download the original data again	`python src/download_data.py`
Run the app	`streamlit run app/streamlit_app.py`
Run the tests	`python -m pytest`
Re-run a notebook	open it in VS Code or Jupyter and run all cells, in order 01, 02, 03

The notebooks must be run from the `notebooks/` folder (VS Code does this by default). Notebook 01 creates `data/processed/nhanes_cohort.csv`, which notebook 03 needs. Notebook 03 creates the files in `models/`, which the app needs. All random steps use `random_state = 42`, so re-running gives the same results.

5. Pipeline overview

1. **Cohort** (`src/nhanes.py`): 10 NHANES files joined on SEQN; answer codes 7 and 9 set to missing; adults 20+, examined, not pregnant, with both lab results, and not already told of kidney disease. 15,560 participants become 7,363.
2. **Label**: CKD = eGFR < 60 (CKD-EPI 2021, no race) or urine ACR >= 30 mg/g. Lab values are used only for the label.
3. **Features**: 13 Tier 3 features, listed in `documentation/data_dictionary.md`.
4. **Split**: 80/20, stratified on the label, `random_state = 42`, saved in the `split` column.
5. **Preprocessing** (`modeling.make_preprocessor`), identical for all models and fit only on training data: numeric features median-imputed and standardized; yes/no features filled with the most common answer; categorical features filled with the most common answer and one-hot encoded.
6. **Models** (`modeling.make_models`): Logistic Regression, Random Forest, and Histogram Gradient Boosting, all with `class_weight="balanced"`.
7. **Tuning** (`modeling.tune`): grid search with 5-fold stratified cross-validation, scored by F2. The decision threshold is tuned for F2 inside each training fold with `TunedThresholdClassifierCV`.

8. **Selection** (`modeling.select_model`): highest mean CV F2, with the pre-registered tie-break toward the simpler model.
9. **Final test**: the selected model is scored once on the test set.
10. **Saved model**: `models/ckd_screening_pipeline.joblib` holds the whole fitted pipeline (preprocessing, model and threshold). `models/model_card.json` holds its settings, results and the risk table used by the app.

The reasons behind each choice are in `documentation/preregistration.md` and in the notebooks.

6. Input format

The app has a form for one patient and a CSV upload for several. Both use the same checks (`src/screening.py`).

CSV columns (a template can be downloaded from the app):

Column	Accepted values
age	number, 20 to 120 years (above 80 is treated as 80)
sex	female, male
weight_kg	number, 20 to 300
height_cm	number, 100 to 230
waist_cm	number, 40 to 200, or empty
sbp	number, 60 to 260 mmHg
dbp	number, 30 to 150 mmHg, lower than sbp
told_htn, bp_meds, high_chol, any_cvd, fam_diabetes	yes, no, dont_know (empty counts as dont_know)
diabetes	no, borderline, yes, dont_know
smoking	never, former, current, dont_know

BMI is calculated from weight and height, and rejected if the result is outside 10 to 100. Rows with errors are skipped and listed with the reason; the other rows are still screened.

7. Application guide

11. Open the app (deployed URL in the README, or run it locally).
12. In **One patient**, fill in the fields and press **Screen this patient**. Waist and the yes/no questions may be left as "Don't know".
13. Read the result:

- **Refer for kidney testing**: the score is at or above the model's threshold.
- **No referral indicated**: the score is below the threshold. This does not rule out CKD.
- **Screening score**: the raw model output. It is not a probability, because the model was trained with balanced class weights, which push scores up.
- **Similar patients with CKD**: the share of training patients with a similar score who had CKD.

14. For many patients, use Upload a CSV. If the clinic can only test some people, enter Tests available first: only that many patients are referred, highest scores first, and every row gets a Priority (1 = highest score). Then download the results.
15. **Inputs and units** lists every field and its accepted range.

8. Model limitations

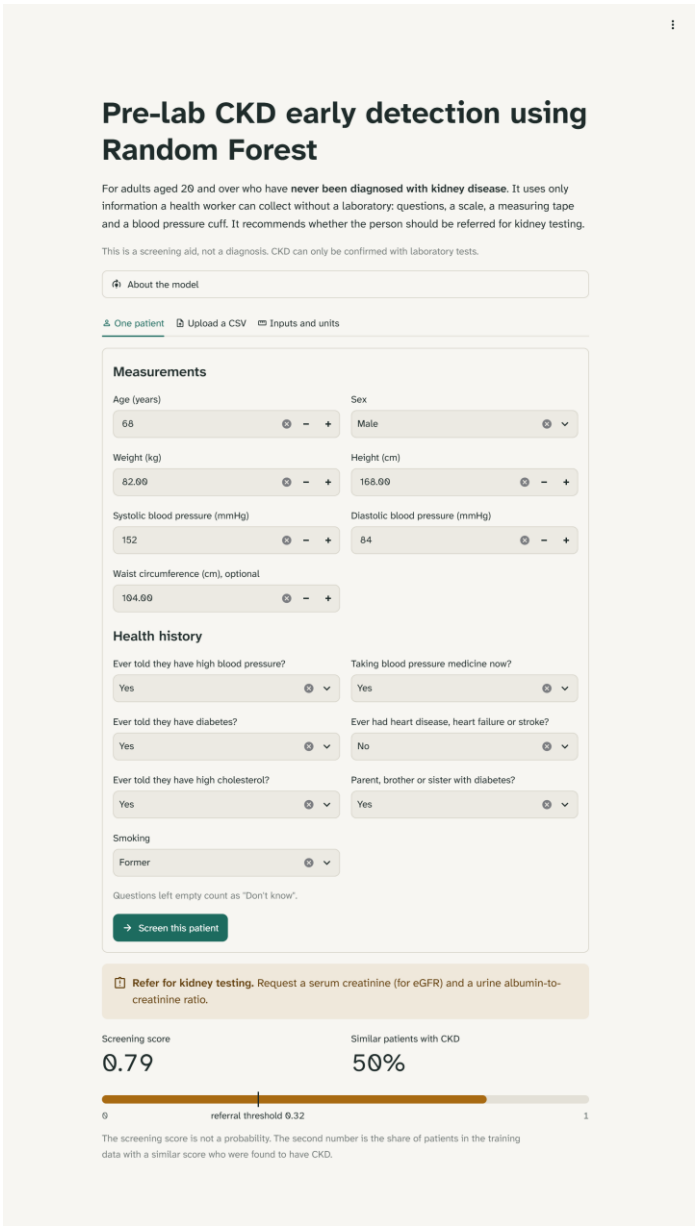
- Trained and tested on a US population. It has not been validated on Filipino patients. The closest subgroup available (Non-Hispanic Asian) is small.
- The CKD label comes from a single measurement, while the clinical definition requires the abnormality to last more than 3 months. Some temporary albuminuria is labeled as CKD.
- Age is partly built into the label, because the eGFR equation contains age. The model leans heavily on age.
- People who were already diagnosed were excluded, so the model is only meant for people who have never been tested.
- A "no referral" result does not rule out CKD. The model misses some patients, especially younger adults with reduced kidney function.
- NHANES survey weights were not used, so results describe the sample, not the US population.
- On F2 the model is tied with a simple risk-factor rule (test difference 0.003, 95% CI -0.017 to 0.024). Its advantage is ranking when tests are limited, and that is a post-hoc finding (notebook 03, section 11).

9. Troubleshooting

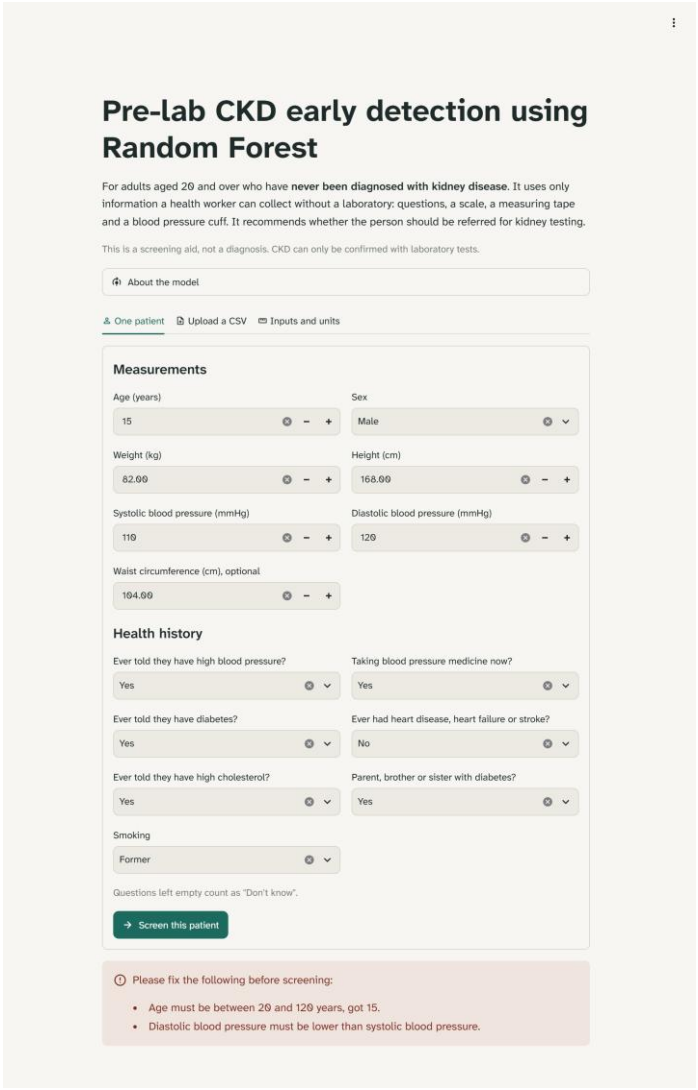
Problem	Fix
`Activate.ps1 cannot be loaded because running scripts is disabled`	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again
`ModuleNotFoundError` for a package	Activate the virtual environment, then `pip install -r requirements.txt`
`ModuleNotFoundError: No module named 'nhanes'` in a notebook	Run the notebook from the `notebooks/` folder, so the project root is one level up
App says the model files could not be loaded	Make sure `models/ckd_screening_pipeline.joblib` and `models/model_card.json` exist. Re-run notebook 03 if needed.
Warning about an unpickling version mismatch	Install the exact versions in `requirements.txt`; the model was saved with the scikit-learn version listed in `models/model_card.json`
`FileNotFoundError` for `nhanes_cohort.csv`	Run notebook 01 first
Data download fails	Check the internet connection. The files are also already in `data/original/`.

10. Screenshots

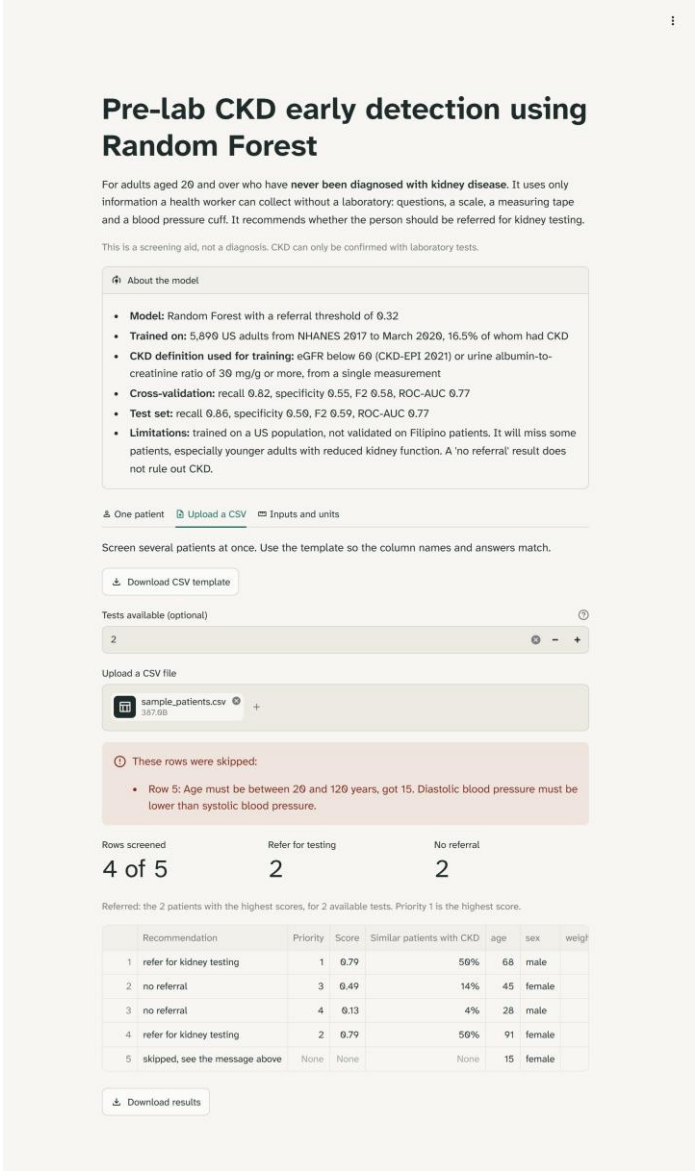
All screenshots are in documentation/screenshots/. They were taken from the app running locally with the five sample patients in Appendix B of the paper.



One patient: a valid input with a referral recommendation.



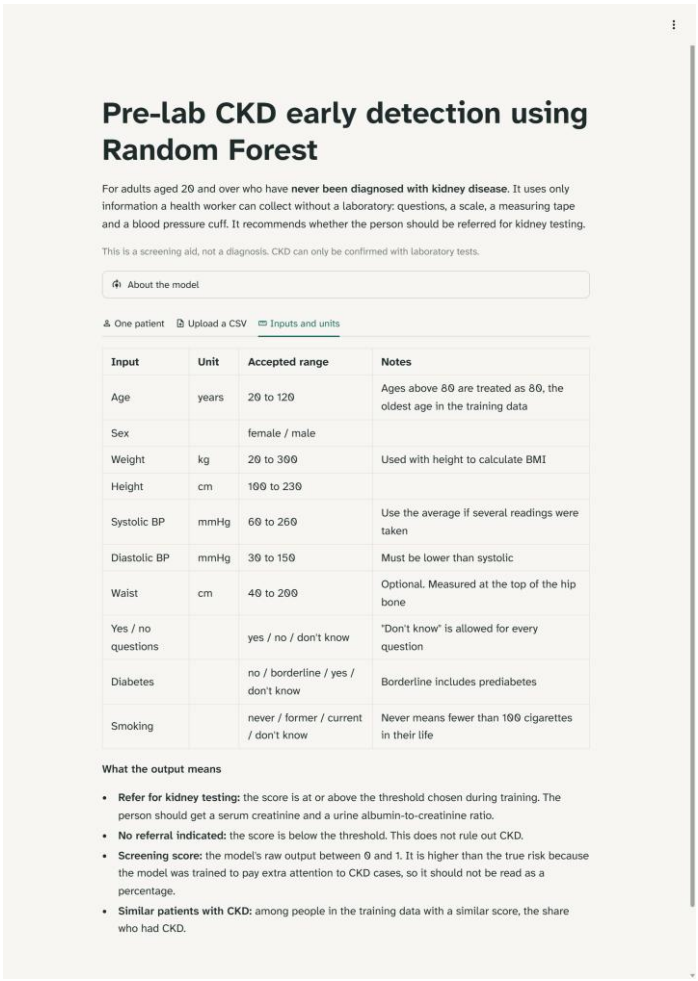
One patient: an out-of-range age and a diastolic pressure above systolic are rejected with a message.



Upload a CSV with 2 tests available: the two highest scores are referred, row 5 is skipped with the reason. The About the model panel is open.

	Recommendation	Priority	Score	Similar patients with CKD	age	sex	weigh
1	refer for kidney testing	1	0.79	50%	68	male	
2	no referral	3	0.49	14%	45	female	
3	no referral	4	0.13	4%	28	male	
4	refer for kidney testing	2	0.79	50%	91	female	
5	skipped, see the message above	None	None	None	15	female	

Upload a CSV: the result columns of the same table, including each row's priority.



Inputs and units tab.